



UNIVERSIDAD SERGIO ARBOLEDA

CAPITULO 1 LIBRO

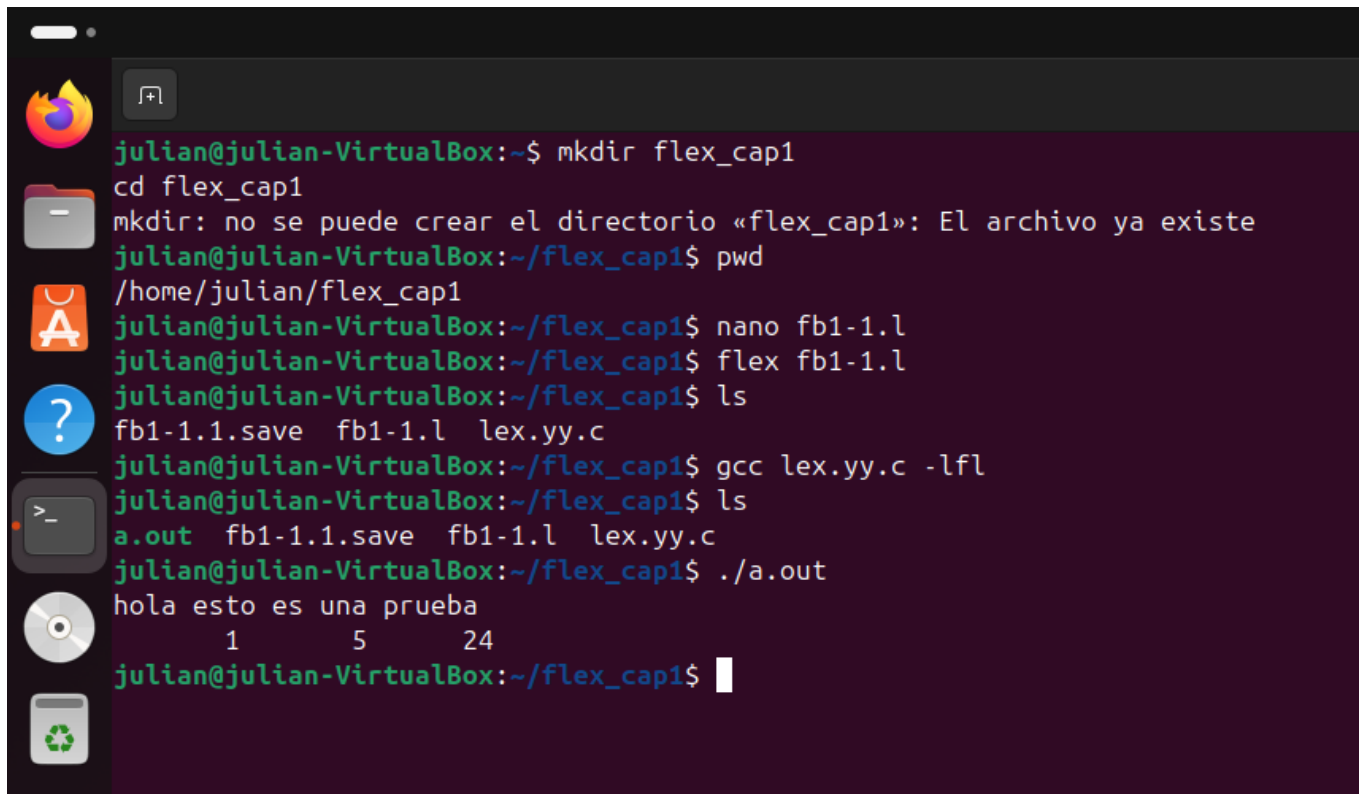
DOCENTE

JOAQUIN SANCHEZ CIFUENTES

PRESENTADO POR

JULIAN CAMILO CARO HERRERA

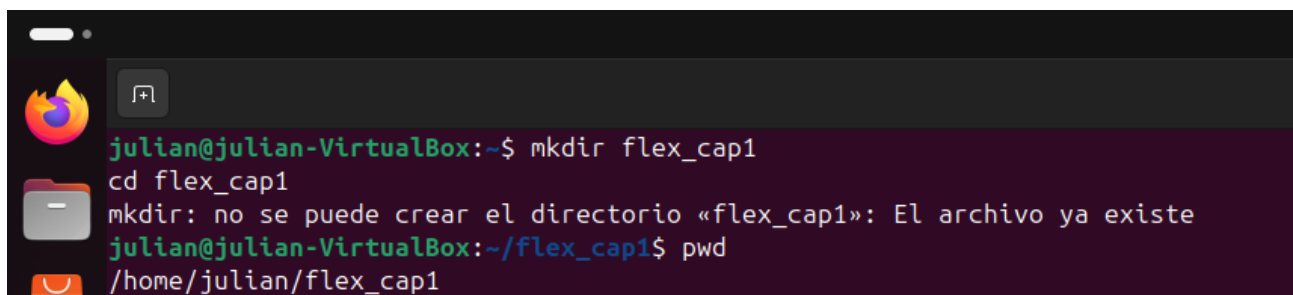
EJEMPLO 1

A terminal window with a dark purple background and a sidebar on the left containing icons for a web browser, file manager, application store, help, terminal, disk, and trash. The terminal text shows the following commands and output:

```
julian@julian-VirtualBox:~$ mkdir flex_cap1
cd flex_cap1
mkdir: no se puede crear el directorio «flex_cap1»: El archivo ya existe
julian@julian-VirtualBox:~/flex_cap1$ pwd
/home/julian/flex_cap1
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ ls
fb1-1.1.save  fb1-1.l  lex.yy.c
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ls
a.out  fb1-1.1.save  fb1-1.l  lex.yy.c
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hola esto es una prueba
      1      5      24
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 1 (Ejercicio completo)

En la figura anterior se encuentra completo el ejemplo 1 del libro, a continuación se explicara el paso a paso de como se hizo. Después de observar este proceso los demás ejercicios no se profundizara mucho en comandos, sino mas en ejecución.

A terminal window showing the first three commands of the example:

```
julian@julian-VirtualBox:~$ mkdir flex_cap1
cd flex_cap1
mkdir: no se puede crear el directorio «flex_cap1»: El archivo ya existe
julian@julian-VirtualBox:~/flex_cap1$ pwd
/home/julian/flex_cap1
```

Figura 2(creación de la carpeta)

En la figura anterior se muestran las primeras líneas de comandos, en las cuales se crea la carpeta con el comando `mkdir flex_cap1`(es el nombre de la carpeta) con el `cd flex_cap1` se ubica en dicho directorio para empezar a trabajar. Con el comando `pwd` se verifica que se esta dentro del directorio

```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-1.l
```

Figura 3 (creación del archivo)

Con el comando nano se crea el archivo con el nombre que se necesite, en este caso se utilizó el mismo con el que se hizo el ejemplo del libro (fb1-1.l)

```
%{
#include <string.h>
#include <stdio.h>

int chars = 0;
int words = 0;
int lines = 0;
}%

%%

[a-zA-Z]+ { words++; chars += strlen(yytext); }
\n      { chars++; lines++; }
.       { chars++; }

%%

int main()
{
    yylex();
    printf("%d%d%d\n", lines, words, chars);
}
```

Figura 4 (nano fb1-1.l)

Ya ubicado en el nano se copió exactamente el mismo código que aparecía en el libro, luego de digitarlo se guarda y se sale del editor nano. El código implementa un analizador léxico con Flex que cuenta caracteres, palabras y líneas de un texto de entrada. Definiendo reglas para identificar secuencias de letras, saltos de línea y cualquier otro carácter, acumulando estadísticas que luego se imprimen en la función main.

```
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ ls
fb1-1.1.save fb1-1.l lex.yy.c
```

Figura 5 (Scanner flex)

Luego de haber hecho el archivo nano, se necesitará realizar el scanner a diferencia del otro fb1-1.l se usará el flex el cual es el que permite hacer el scanner. Luego de poner dicho comando se utilizará ls para revisar que haya quedado correctamente deberían de aparecer fb1-1.l lex.yy.c, los cuales se observan en la figura 5

```
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ls
a.out fb1-1.1.save fb1-1.l lex.yy.c
```

Figura 6(Compilacion)

Para poder compilar lo que ya se hizo , y lo importante que era verificar que haya quedado bien el scanner porque se compilara lex.yy.c, con el comando gcc (lex.yy.c) -lfl, después de ejecutar el comando con el ls se verificara que haya salido el a.out para poder ejecutarlo

```
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hola esto es una prueba
      1      5     24
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 7(ejecución y prueba)

Despues de haber verificado que si haya quedado el a.out, se realizara el comando (./a.out) con esto se ejecutara y se podrá ingresar desde teclado la respectiva prueba, en este caso se digito “hola esto es una prueba”, esto debería dar como resultado **líneas, palabras , caracteres**, como se puede observar hay una línea, cinco palabras y 24 caracteres. Lo que quiere decir que se realizo con éxito el primer ejemplo

EJEMPLO 2

```
julian@julian-VirtualBox:~$ cd flex_cap1
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-2.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-2.l
julian@julian-VirtualBox:~/flex_cap1$ ls
a.out fb1-1.1.save fb1-1.l fb1-2.l lex.yy.c
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
The colour of the theatre centre has flavour
The color of the theater center has flavor
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 8 (Ejemplo 2 completo)

```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-2.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-2.l
julian@julian-VirtualBox:~/flex_cap1$ ls
a.out  fb1-1.1.save  fb1-1.l  fb1-2.l  lex.yy.c
```

Figura 9 (creación del archivo y scanner)

Como se explico con el ejemplo 1 se realizo el mismo proceso para la creación del scanner y el archivo nano como se vera en la siguiente figura

```
GNU nano 7.2 fb1-2.1 *
%{
#include <stdio.h>
}%

%%

colour    printf("color");
flavour   printf("flavor");
centre    printf("center");
theatre   printf("theater");

\n        printf("\n");
.          printf("%s", yytext);

%%

int main()
{
    yylex();
}
```

Figura 10(código nano)

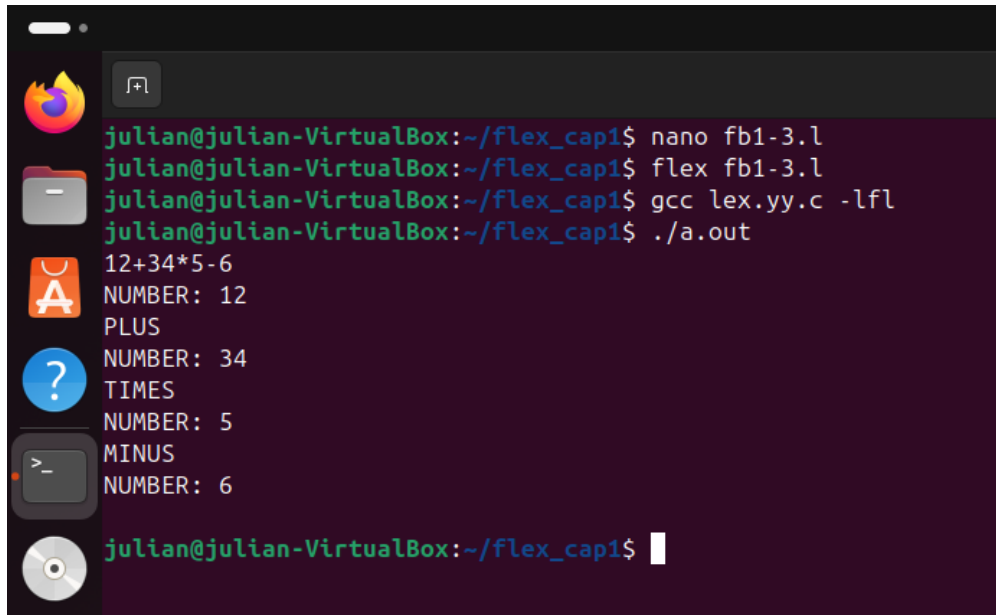
El código es un analizador léxico con Flex que transforma palabras en inglés británico a inglés americano. Las reglas especifican sustituciones directas (ej. colour → color) y mantienen el resto del texto sin cambios. La función main inicia el análisis con yylex.

```
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
The colour of the theatre centre has flavour
The color of the theater center has flavor
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 11(compilación y ejecucion)

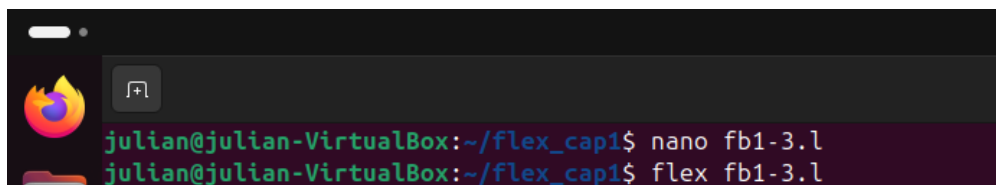
Luego de haber creado todo correctamente, se compilara con el mismo comando y también la ejecución, se digitara por consola la primera línea que esta en ingles británico y como resultado arrojará la traducción a el ingles (Estados Unidos)

EJEMPLO 3



```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-3.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-3.l
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
12+34*5-6
NUMBER: 12
PLUS
NUMBER: 34
TIMES
NUMBER: 5
MINUS
NUMBER: 6
julian@julian-VirtualBox:~/flex_cap1$
```

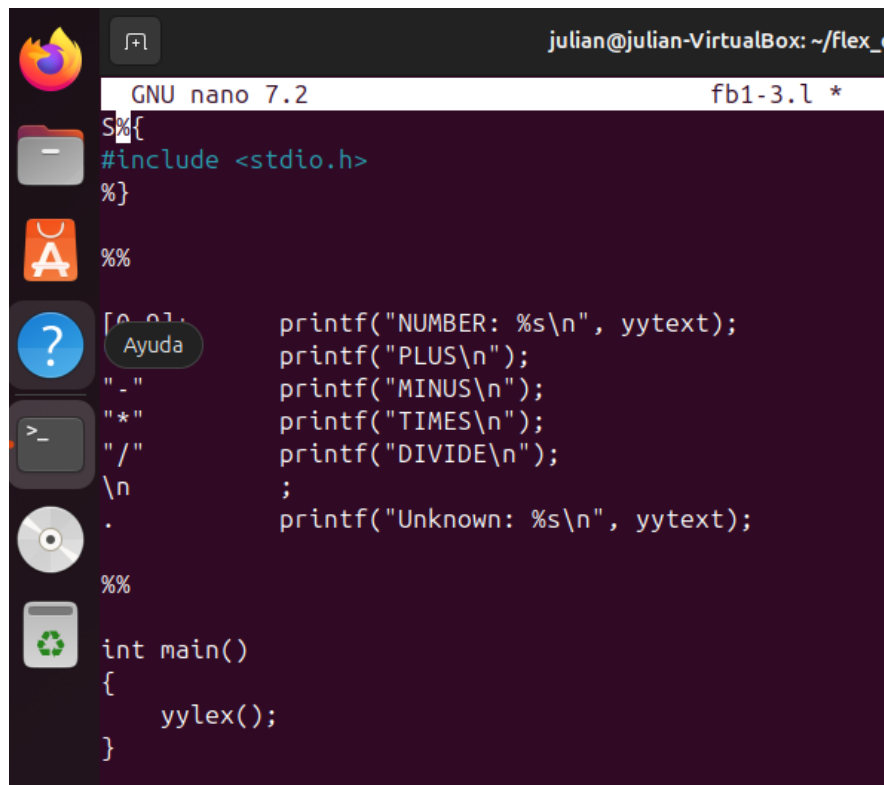
Figura 12 (Ejemplo 3 completo)



```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-3.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-3.l
```

Figura 13 (creación del archivo y scanner)

Como se explica con el ejemplo 1 se realizó el mismo proceso para la creación del scanner y el archivo nano como se verá en la siguiente figura

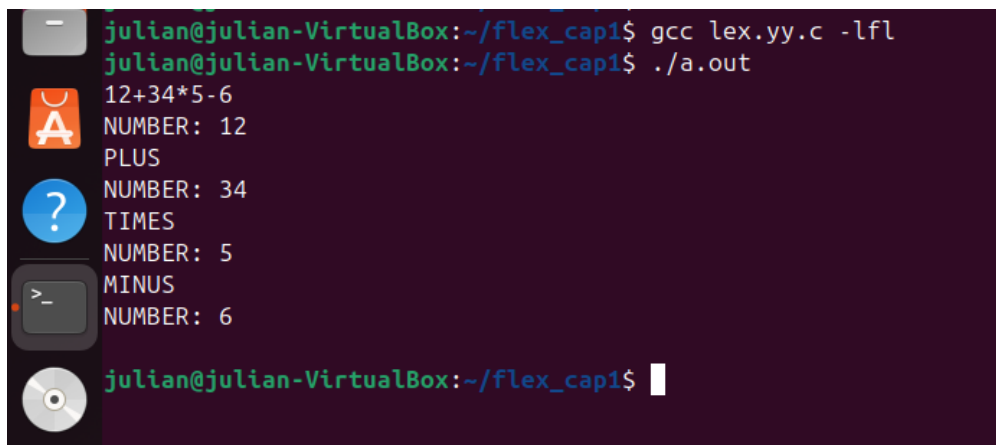


```
GNU nano 7.2 fb1-3.1 *
#include <stdio.h>
%%
printf("NUMBER: %s\n", yytext);
printf("PLUS\n");
printf("MINUS\n");
printf("TIMES\n");
printf("DIVIDE\n");
;
printf("Unknown: %s\n", yytext);

int main()
{
    yylex();
}
```

Figura 14 (código nano)

El código es un analizador léxico que reconoce números y operadores aritméticos básicos (+, -, *, /). Cada token identificado se imprime con un mensaje específico, mientras que cualquier otro carácter se clasifica como desconocido.



```
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
12+34*5-6
NUMBER: 12
PLUS
NUMBER: 34
TIMES
NUMBER: 5
MINUS
NUMBER: 6
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 15 (compilacion y ejecucion)

Se ingreso por teclado la operación 12+34*5-6, como resultado se espera la lectura de los números y los operadores básicos, en caso de no haber ninguno arrojará como desconocido como se ve en la siguiente figura

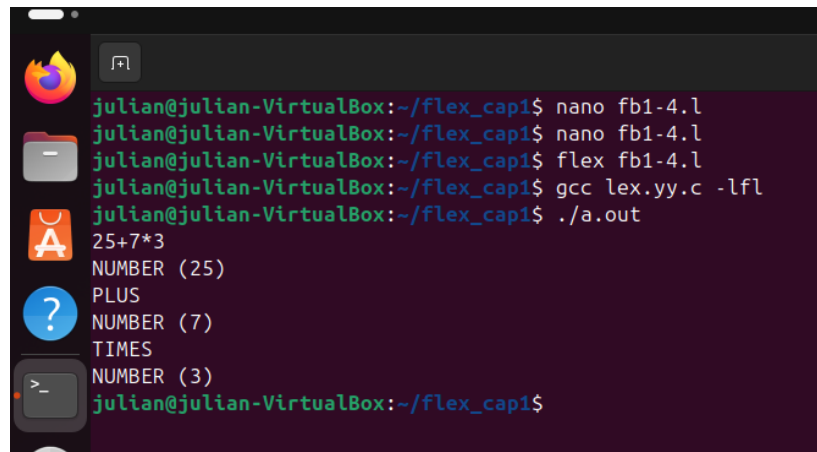
```

julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hoadkawodjqe
Unknown: h
Unknown: o
Unknown: a
Unknown: d
Unknown: k
Unknown: a
Unknown: w
Unknown: o
Unknown: d
Unknown: j
Unknown: q
Unknown: e
julian@julian-VirtualBox:~/flex_cap1$ █

```

Figura 16 (Caso de uso)

EJEMPLO 4

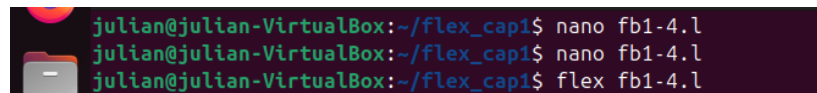


```

julian@julian-VirtualBox:~/flex_cap1$ nano fb1-4.1
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-4.1
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-4.1
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
25+7*3
NUMBER (25)
PLUS
NUMBER (7)
TIMES
NUMBER (3)
julian@julian-VirtualBox:~/flex_cap1$

```

Figura 17 (Ejemplo 4 completo)



```

julian@julian-VirtualBox:~/flex_cap1$ nano fb1-4.1
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-4.1
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-4.1

```

Figura 18 (creación del archivo y el scanner)


```
GNU nano 7.2
%{
#include <stdio.h>
#include <stdlib.h>

int yylval;
}%

%%
[0-9]+      { yylval = atoi(yytext);
              printf("NUMBER (%d)\n", yylval); }

"+"         printf("PLUS\n");
"-"         printf("MINUS\n");
"*"         printf("TIMES\n");
"/"         printf("DIVIDE\n");

\n          ;
.           printf("Unknown: %s\n", yytext);

%%

int main()
{
    yylex();
}
```

Figura 19 (Nano)

reconoce números y operadores aritméticos. Los números se convierten a enteros se imprimen junto con su valor, mientras que los operadores básicos (+, -, *, /) se identifican con etiquetas específicas. Cualquier otro carácter se marca como desconocido.

```
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
25+7*3
NUMBER (25)
PLUS
NUMBER (7)
TIMES
NUMBER (3)
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 20 (compilación y ejecución)

Por teclado se ingresó la siguiente operación $25+7*3$, muy parecido al anterior ejemplo, sino que en este caso salen los números al lado. Igual que el anterior al ingresar otro carácter sale desconocido.

```

julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hoadkawodjqe
Unknown: h
Unknown: o
Unknown: a
Unknown: d
Unknown: k
Unknown: a
Unknown: w
Unknown: o
Unknown: d
Unknown: j
Unknown: q
Unknown: e

```

Figura 21(caso de uso)

EJEMPLO 5

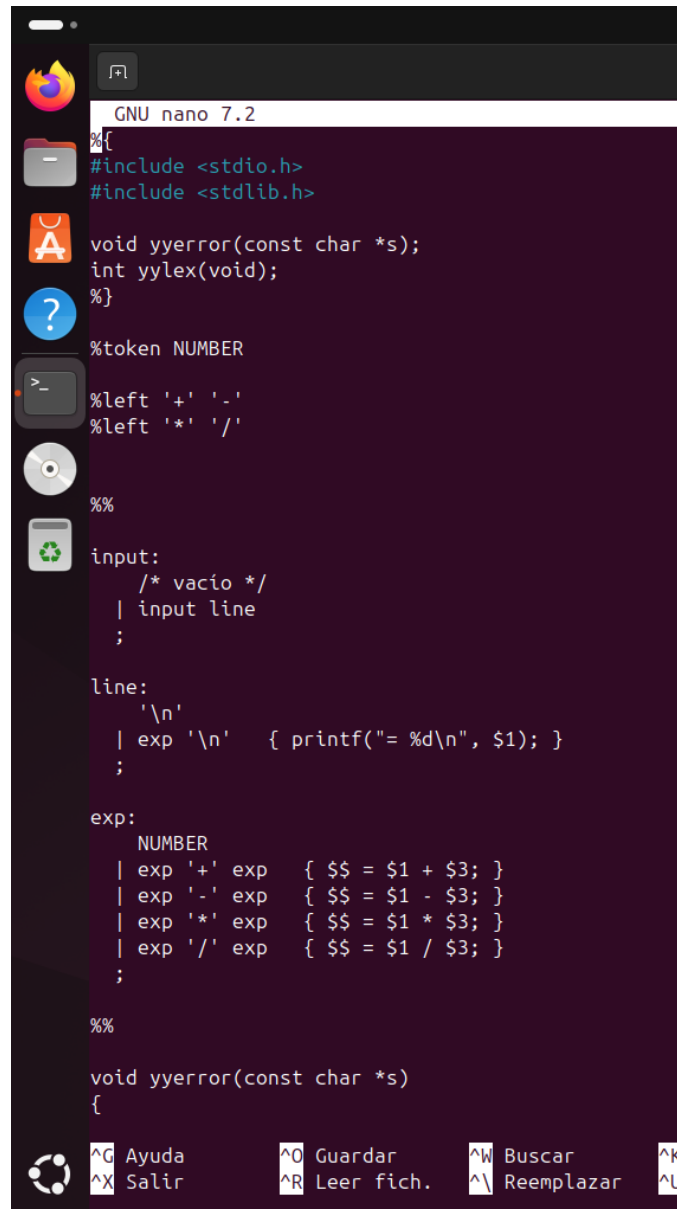
```

julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.l
julian@julian-VirtualBox:~/flex_cap1$ bison -d fb1-5.y

```

Figura 22 (creación archivo,scanner y parser)

En este ejemplo a diferencia de los anteriores se creara un parser con el siguiente comando :
(bison -d fb1-5.y), que genera (fb1-5.tab.c) (fb1-5.tab.h)



```
GNU nano 7.2
%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex(void);
}%

%token NUMBER

%left '+' '-'
%left '*' '/'

%%

input:
    /* vacio */
    | input line
    ;

line:
    '\n'
    | exp '\n' { printf("= %d\n", $1); }
    ;

exp:
    NUMBER
    | exp '+' exp { $$ = $1 + $3; }
    | exp '-' exp { $$ = $1 - $3; }
    | exp '*' exp { $$ = $1 * $3; }
    | exp '/' exp { $$ = $1 / $3; }
    ;

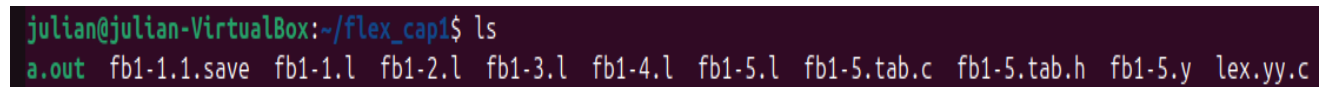
%%

void yyerror(const char *s)
{

^G Ayuda    ^O Guardar  ^W Buscar   ^K
^X Salir    ^R Leer fich. ^_ Reemplazar ^U
```

Figura 23 (nano)

Evalúa expresiones aritméticas. Se establecen reglas gramaticales que reconocen números y operadores (+, -, *, /), aplicando la precedencia correspondiente. Al procesar una línea, el parser calcula el resultado y lo imprime.



```
julian@julian-VirtualBox:~/flex_cap1$ ls
a.out fb1-1.1.save fb1-1.l fb1-2.l fb1-3.l fb1-4.l fb1-5.l fb1-5.tab.c fb1-5.tab.h fb1-5.y lex.yy.c
```

Figura 24(verificación de los archivos)

```
julian@julian-VirtualBox:~/flex_cap1$ gcc fb1-5.tab.c lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
3+4
= 7
```

Figura 25(resultado)

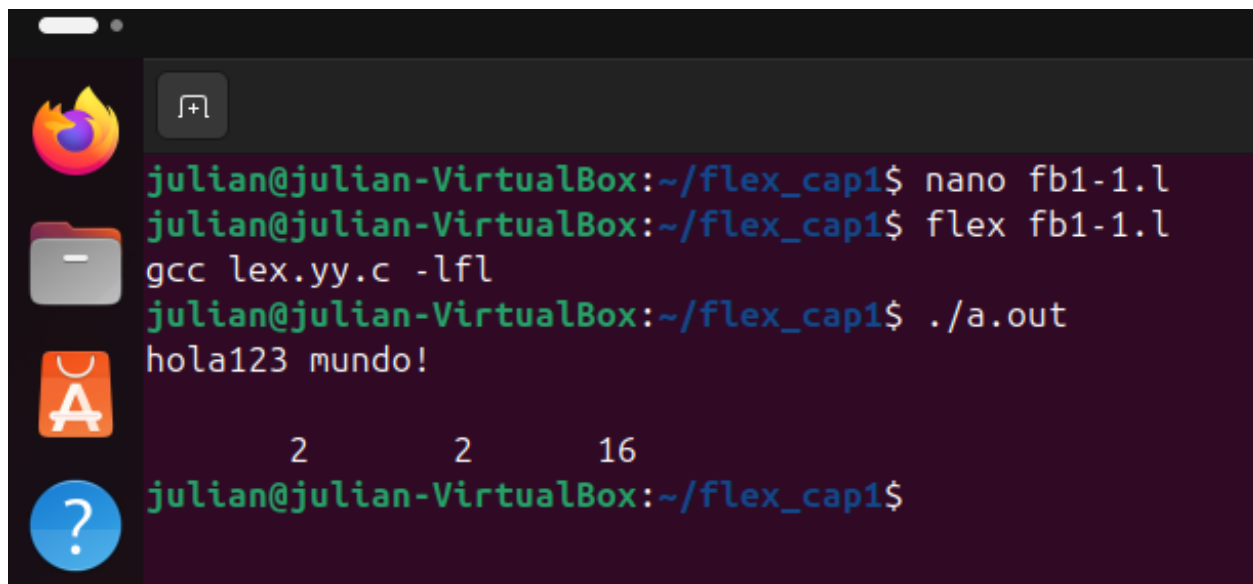
```
10*2+5
= 25
```

Figura 26(resultado)

Se pudo verificar si cumple con el orden para realizar las operaciones y dar el resultado correcto

EJERCICIO 1

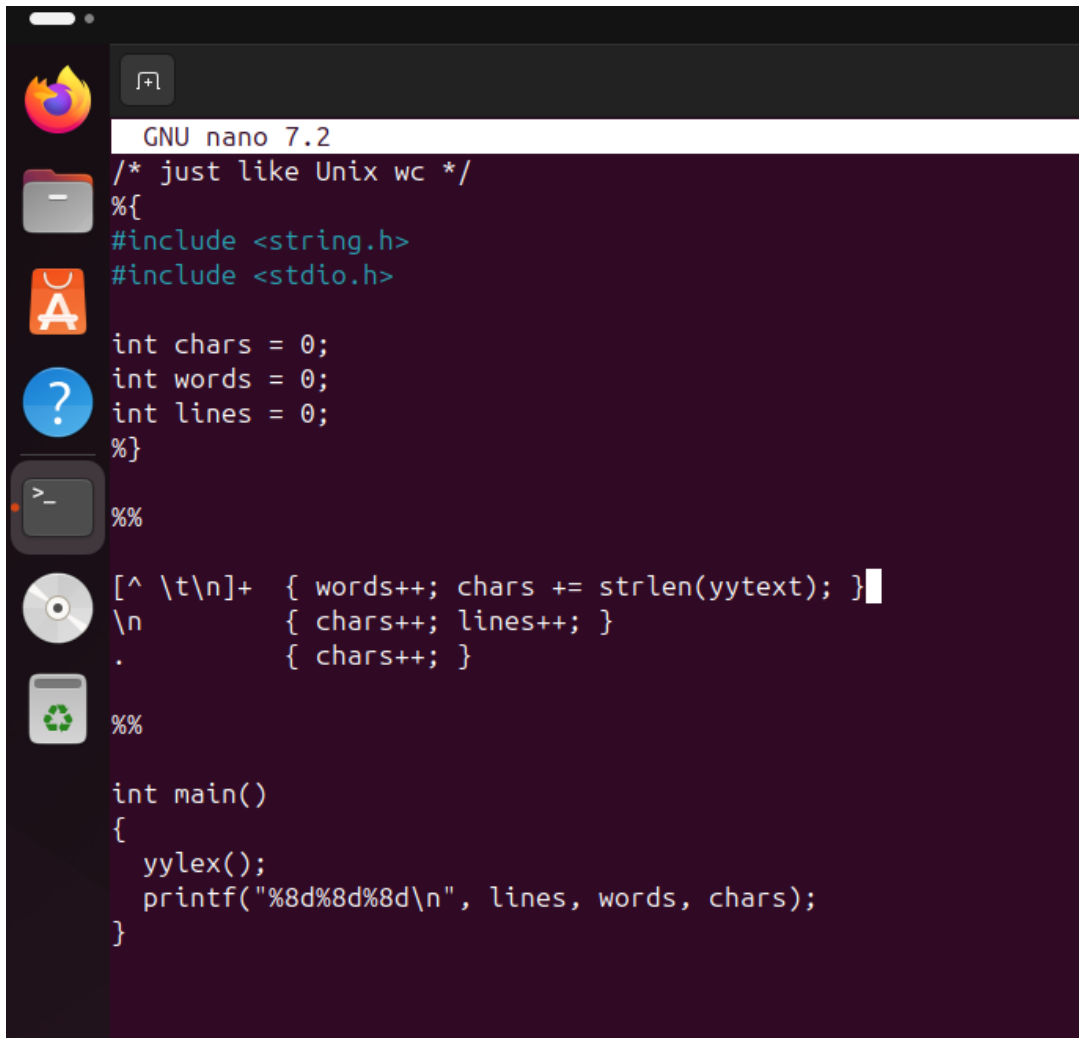
(El ejercicio pide mejorar la definición de palabra para que sea más realista) teniendo en cuenta el ejemplo 1



```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-1.l
gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hola123 mundo!
      2      2      16
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 27 (Ejercicio 1 completo)

Con base al ejemplo 1 solo se tiene que modificar el código para que cumpla con los requerimientos (realismo)



```
GNU nano 7.2
/* just like Unix wc */
%{
#include <string.h>
#include <stdio.h>

int chars = 0;
int words = 0;
int lines = 0;
}%

%%

[^\t\n]+ { words++; chars += strlen(yytext); }
\n      { chars++; lines++; }
.       { chars++; }

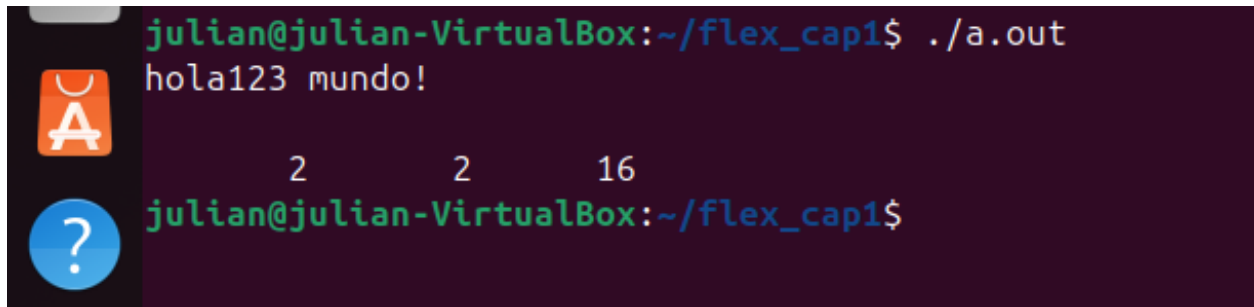
%%

int main()
{
    yylex();
    printf("%8d%8d%8d\n", lines, words, chars);
}
```

Figura 28(Correccion del nano)

[a-zA-Z]+: solo contaba secuencias de letras.

Ahora: [^\t\n]+: cuenta cualquier secuencia de caracteres que no sea espacio, tabulación o salto de línea.

A terminal window with a dark purple background. The prompt is 'julian@julian-VirtualBox:~/flex_cap1\$'. The command './a.out' is entered, and the output is 'hola123 mundo!'. Below the output, the numbers '2', '2', and '16' are displayed, corresponding to the counts of lines, words, and characters respectively. The prompt is then 'julian@julian-VirtualBox:~/flex_cap1\$'.

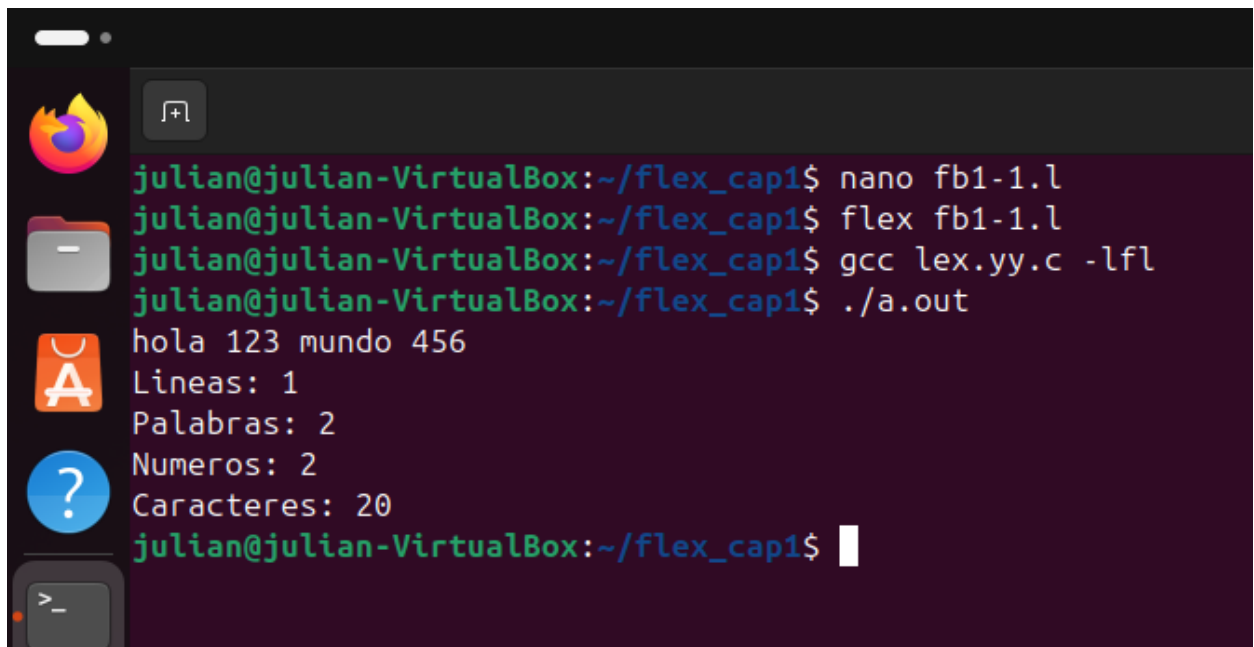
```
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hola123 mundo!
2      2      16
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 29(resultado)

Con el ejemplo 1 antes solo contaba el hola mundo, ahora las cuenta como palabras completas, algo a tener en cuenta es que salen 2 líneas porque hay un salto de línea, hola123 como una palabra y mundo como otra y cuenta los caracteres correctamente

EJERCICIO 2

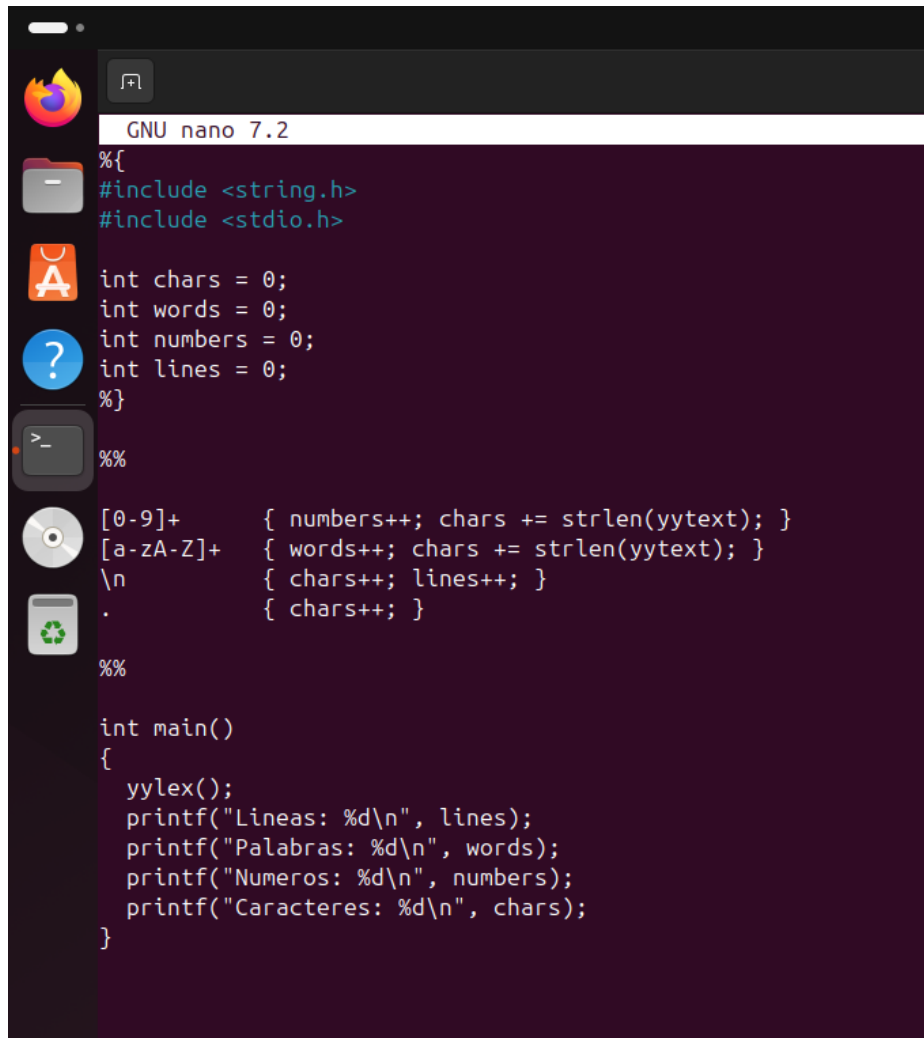
mejorar el contador para que también cuente números separados de palabras.

A terminal window with a dark purple background. The prompt is 'julian@julian-VirtualBox:~/flex_cap1\$'. The command 'nano fb1-1.l' is entered, followed by 'flex fb1-1.l', then 'gcc lex.yy.c -lfl', and finally './a.out'. The output is 'hola 123 mundo 456'. Below the output, the statistics are displayed: 'Lineas: 1', 'Palabras: 2', 'Numeros: 2', and 'Caracteres: 20'. The prompt is then 'julian@julian-VirtualBox:~/flex_cap1\$'.

```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hola 123 mundo 456
Lineas: 1
Palabras: 2
Numeros: 2
Caracteres: 20
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 30 (Ejercicio completo 2)

Se modificara el ejemplo 2 para que cuente separado palabras de números



```
GNU nano 7.2
%{
#include <string.h>
#include <stdio.h>

int chars = 0;
int words = 0;
int numbers = 0;
int lines = 0;
}%

%%

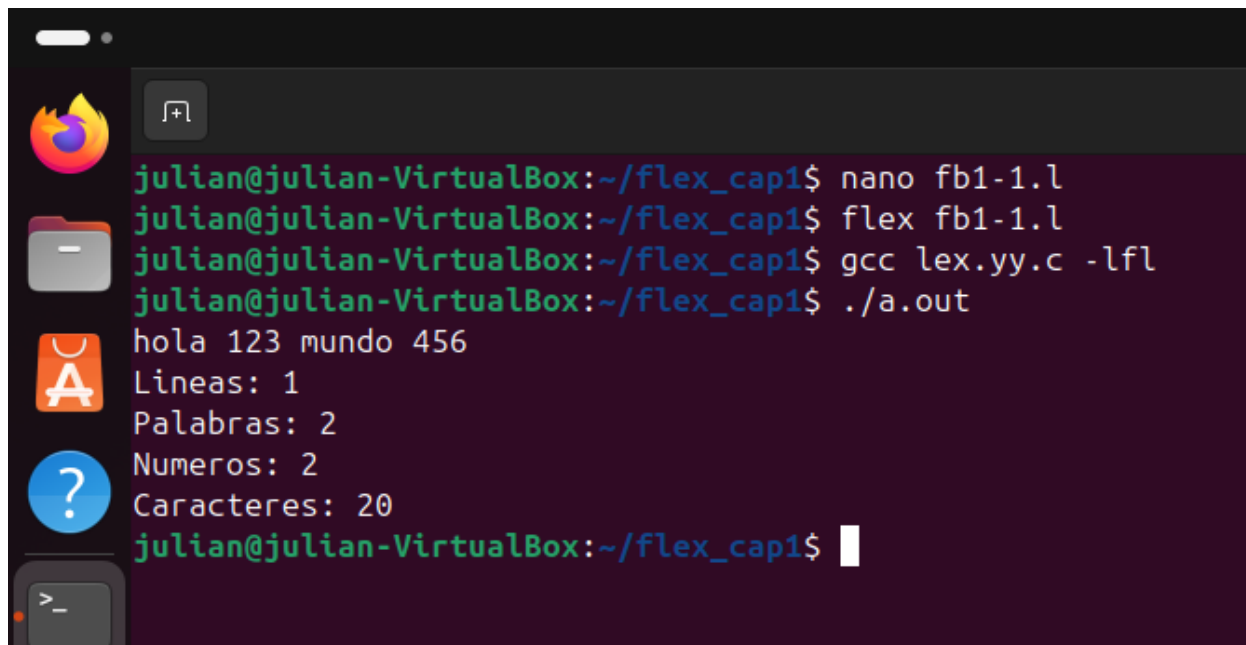
[0-9]+      { numbers++; chars += strlen(yytext); }
[a-zA-Z]+   { words++; chars += strlen(yytext); }
\n          { chars++; lines++; }
.           { chars++; }

%%

int main()
{
    yylex();
    printf("Lineas: %d\n", lines);
    printf("Palabras: %d\n", words);
    printf("Numeros: %d\n", numbers);
    printf("Caracteres: %d\n", chars);
}
```

Figura 31 (modificación nano)

En comparación con el de el ejemplo se tienen en cuenta o se modifica la regla para que los números queden antes de la palabra. de esta forma quedan separados y al ejecutar el main saldrán a parte. El orden es lo mas importante para resolver este ejercicio



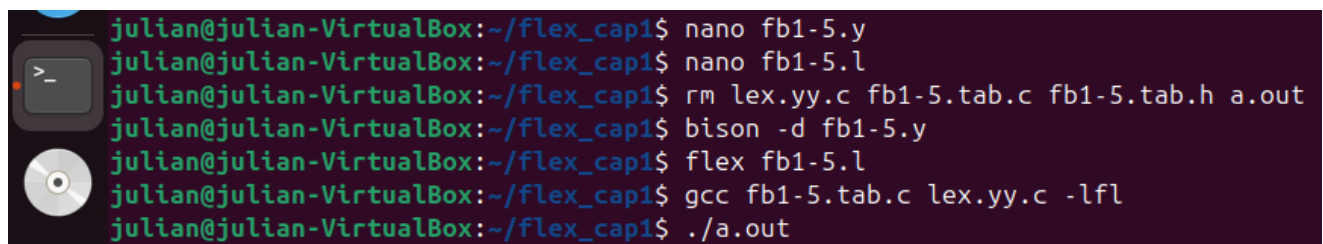
```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-1.l
julian@julian-VirtualBox:~/flex_cap1$ gcc lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
hola 123 mundo 456
Lineas: 1
Palabras: 2
Numeros: 2
Caracteres: 20
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 32 (resultados)

Efectivamente salió aparte letras y números

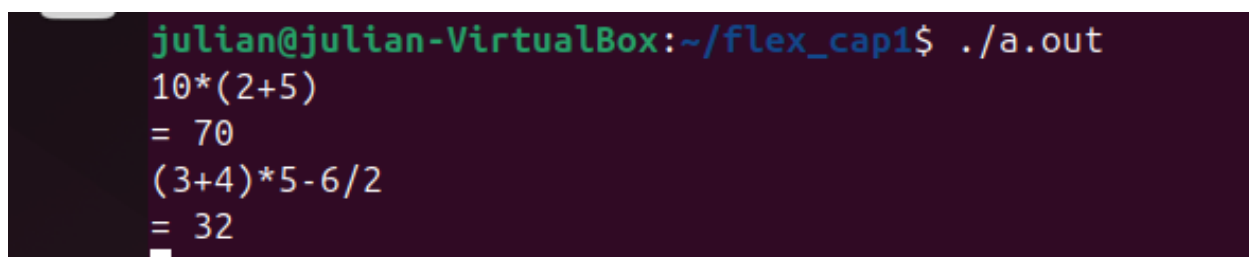
Ejercicio 3

mejorar la calculadora para que soporte paréntesis correctamente.



```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.l
julian@julian-VirtualBox:~/flex_cap1$ rm lex.yy.c fb1-5.tab.c fb1-5.tab.h a.out
julian@julian-VirtualBox:~/flex_cap1$ bison -d fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-5.l
julian@julian-VirtualBox:~/flex_cap1$ gcc fb1-5.tab.c lex.yy.c -lfl
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
```

Figura 33(ejercicio 3 completo)



```
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
10*(2+5)
= 70
(3+4)*5-6/2
= 32
```

Figura 34(resultados)


```
GNU nano 7.2
%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex(void);
}%

%token NUMBER

%%

input:
    /* vacio */
    | input line
    ;

line:
    '\n'
    | exp '\n'    { printf("= %d\\n", $1); }
    ;

exp:
    exp '+' term  { $$ = $1 + $3; }
    | exp '-' term  { $$ = $1 - $3; }
    | term
    ;

term:
    term '*' factor  { $$ = $1 * $3; }
    | term '/' factor  { $$ = $1 / $3; }
    | factor
    ;

factor:
    NUMBER
    | '(' exp ')'    { $$ = $2; }

^G Ayuda      ^O Guardar    ^W Buscar
^X Salir      ^R Leer fich. ^\ Reemplazar
```

Figura 35(mejora nano)

Si se define todo como `exp: exp '+' exp`, se depende de las declaraciones de precedencia (`%left`, `%right`) para resolver ambigüedades.

- En cambio, al estructurar jerárquicamente (`exp` $\rightarrow$ `term` $\rightarrow$ `factor` $\rightarrow$ `NUMBER`), la precedencia ya está implícita en la propia gramática:
- **exp** maneja suma y resta, **term** maneja multiplicación y división, **factor** maneja números y paréntesis.

```
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
10*(2+5)
= 70
(3+4)*5-6/2
= 32
```

Figura 36(resultados)

EJERCICIO 4

Agregar operador de potencia

```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.l
julian@julian-VirtualBox:~/flex_cap1$ rm lex.yy.c fb1-5.tab.c fb1-5.tab.h a.out
julian@julian-VirtualBox:~/flex_cap1$ bison -d fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-5.l
julian@julian-VirtualBox:~/flex_cap1$ gcc fb1-5.tab.c lex.yy.c -lfl -lm
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
2^3
= 8
2^3^2
= 512
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 37 (Ejercicio 4 completo)

```
GNU nano 7.2                                fb1-5.l *
%{
#include "fb1-5.tab.h"
#include <stdlib.h>
}%

%%

[0-9]+      { yylval = atoi(yytext); return NUMBER; }
"+"         return '+';
"-"         return '-';
"*"         return '*';
"/"         return '/';
"^"         return '^';
"("         return '(';
")"         return ')';
\n          return '\n';
[ \t]       ;
.           ;

%%
```

Figura 38 (modificación nano)

La gramática se amplió para incluir la operación de potencia mediante la regla factor '^'. Esta modificación, junto con la inclusión de la librería `<stdlib.h>`, permite que el analizador sintáctico procese expresiones con exponentes de manera correcta. La estructura jerárquica de la gramática ($\text{exp} \rightarrow \text{term} \rightarrow \text{factor}$) asegura que la precedencia de las operaciones se respete de forma natural.

```
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
2^3
= 8
2^3^2
= 512
julian@julian-VirtualBox:~/flex_cap1$
```

Figura 39 (resultados)

EJERCICIO 5

Manejo de errores

```
julian@julian-VirtualBox:~/flex_cap1$ nano fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ rm lex.yy.c fb1-5.tab.c fb1-5.tab.h a.out
julian@julian-VirtualBox:~/flex_cap1$ bison -d fb1-5.y
julian@julian-VirtualBox:~/flex_cap1$ flex fb1-5.l
julian@julian-VirtualBox:~/flex_cap1$ gcc fb1-5.tab.c lex.yy.c -lfl -lm
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
2+*3
Error: syntax error
4+5
= 9
```

Figura 40(Ejercicio completo 5)

```
line:
    '\n'
    | exp '\n'      { printf("= %d\n", $1); }
    | error '\n'    { yyerrok; }
    ;
```

Figura 41(Modificacion del nano)

Se agregó la producción error '\n' para permitir la recuperación de errores sintácticos. De esta forma, cuando ocurre un error en una línea, el parser lo descarta y continúa procesando las siguientes expresiones sin finalizar la ejecución.

```
julian@julian-VirtualBox:~/flex_cap1$ ./a.out
2+3
Error: syntax error
4+5
= 9
```

Figura 42(resultados)

EJERCICIO 6

Al ejecutar el (bison -v fb1-5.y) el parser, el bison lo que hace es generar (fb1-5.output), este archivo muestra , las reglas de manera numerada, los estados en los que se encuentra el parser, las transiciones, las acciones, y verificar si se presentan conflictos

```
Gramática
0 $accept: input $end
1 input: ε
2   | input line
3 line: '\n'
4   | exp '\n'
5   | error '\n'
6 exp: exp '+' term
7   | exp '-' term
8   | term
9 term: term '*' power
10  | term '/' power
11  | power
12 power: factor '^' power
13  | factor
14 Factor: NUMBER
15  | '(' exp ')'
```

Terminales con las reglas donde aparecen

```
$end (0) 0
'\n' (10) 3 4 5
'(' (40) 15
')' (41) 15
'*' (42) 9
'+' (43) 6
'-' (45) 7
'/' (47) 10
'^' (94) 12
error (256) 5
```

No terminales con las reglas donde aparecen

```
$accept (12)
  on left: 0
input (13)
  on left: 1 2
  on right: 0 2
line (14)
  on left: 3 4 5
  on right: 2
exp (15)
  on left: 6 7 8
  on right: 4 6 7 15
term (16)
  on left: 9 10 11
  on right: 6 7 8 9 10
power (17)
  on left: 12 13
  on right: 9 10 11 12
factor (18)
  on left: 14 15
  on right: 12 13
```

Estado 0

```
0 $accept: • input $end

$default reduce usando la regla 1 (input)

input ir al estado 1
```

```

Estado 2

0 $accept: input $end •

$default  aceptar

Estado 3

5 line: error • '\n'

'\n' desplazar e ir al estado 12

Estado 4

14 factor: NUMBER •

$default  reduce usando la regla 14 (factor)

Estado 5

3 line: '\n' •

$default  reduce usando la regla 3 (line)

Estado 6

15 factor: '(' • exp ')'

NUMBER desplazar e ir al estado 4
'(' desplazar e ir al estado 6
'*' desplazar e ir al estado 17
 '/' desplazar e ir al estado 18

$default  reduce usando la regla 6 (exp)

Estado 22

7 exp: exp '-' term •
9 term: term • '*' power
10   | term • '/' power

'*' desplazar e ir al estado 17
 '/' desplazar e ir al estado 18

$default  reduce usando la regla 7 (exp)

Estado 23

9 term: term '*' power •

$default  reduce usando la regla 9 (term)

Estado 24

10 term: term '/' power •

$default  reduce usando la regla 10 (term)

Estado 25

12 power: factor '^' power •

$default  reduce usando la regla 12 (power)

```

El parser generado contiene 25 estados

No presenta conflictos

El archivo .output muestra las transiciones y reglas activas en cada estado.